Unit 10: CNN Tutorial

Introduction

The brief asks me to look inside a convolutional neural network (CNN) and test it on images similar and dissimilar to its training objects. CNN Explainer runs Tiny VGG, trained on ten Tiny ImageNet classes to 70.8% top-1 validation accuracy (Wang et al., 2021). I loaded its weights in Colab, so every score I report for this network comes from one run.

The images

My reference group is the ten built-in examples. A synthetic sports car tests a trained class and a synthetic teacup a related one, since tea is not espresso. Five images sit outside the label set: three photos of my cat, our New Year table and a synthetic wrench. Before my first run I wrote down the class I expected for each photo, choosing something of a similar type, shape or colour: red panda for my two ginger cats and koala for my grey one. My expectations for the synthetic images came later, after an earlier trial. Of all seven, only the teacup matched.

What the layers show

Early convolution layers respond to small, simple patterns and later ones combine them into bigger structures, while pooling downsizes each map so there is less to compute (Géron, 2025); in my run, the first pooling layer reduced each 60 × 60 map to 30 × 30. The maps show what each operation does, not what a filter has learned to mean. At the final ReLU layer, 61.8% of values were positive for the built-in examples and 59.7% for the images outside the label set, a measure that barely separated them.

Results

All ten built-in examples were classified correctly, at 0.786 to 0.995, which shows the tool working rather than its accuracy. My sports car came out as ladybug at 0.876, with bell pepper and orange next; all three are red or orange objects, so colour may carry much of the decision, though I did not test this. The wrench scored 0.983 for espresso and the cats 0.795 to 0.916. The teacup was labelled espresso, the label I had expected from its visual similarity. Fur colour did not lead to red panda or koala: all three cats became espresso. Softmax turns the outputs into scores that sum to one (Wang et al., 2021), and with no “unknown” class the highest always wins.

Preparation changed the top class for every photo and for none of the synthetic images. In the browser the blanket photo was pizza; my centre crop said espresso. Replacing the resizing routine in Colab with one step without smoothing also gave pizza (0.886). Whether the browser and Colab used identical input pixels remains untested, and my copy omits the tool’s extra handling of non-square images (Polo Club of Data Science, 2023).

Would a threshold help?

No minimum score separated the groups in this sample. Any threshold high enough to reject the wrench would also reject six of the ten built-in examples. At 0.90, seven built-in examples are kept, while two, one or none of the five outside images pass, depending on the preparation. Spotting inputs from outside the training classes is a separate problem from classifying them (Yang et al., 2024).

Unit 9 and Unit 10

Unit 9 tested a different CNN on the CIFAR-10 test set (Krizhevsky, 2009), which includes cats, and 330 of its 3,609 errors still scored 0.9 or more. This network has

no cat class, and no accuracy measured on its own ten classes could have shown what it does with my cat.

Conclusion

In my governance work, I would not accept a confidence threshold as the safeguard for a model like this. Before sign-off I would ask for two measured rates: how often the abstention rule accepts images from outside the label set, and how often it rejects valid ones. One fixed input pipeline matters as much, because swapping the resizing routine changed the blanket cat’s label.

Evidence from my run

Tables 1 to 5 and Figures 1 and 2 come from my Colab run of 12 November 2026 at 18:58 UTC, in which all 14 code cells completed without errors. The notebook loads the trained weights and built-in examples from the tool's repository, fixed to commit d0971f9 (Polo Club of Data Science, 2023). Table 6 and Figures 3 to 5 come from my CNN Explainer screenshots of the same day. The tool does not say whether its built-in examples were training images, and the three synthetic images were generated for this exercise, not photographed.

Reference group

Table 1: The ten built-in examples, already 64 × 64. Score is the highest of the ten softmax outputs.

Example	Predicted	Score	Correct
lifeboat	lifeboat	0.821	Yes
ladybug	ladybug	0.984	Yes
pizza	pizza	0.991	Yes
bell pepper	bell pepper	0.786	Yes
school bus	school bus	0.995	Yes
koala	koala	0.953	Yes
espresso	espresso	0.880	Yes
red panda	red panda	0.957	Yes
orange	orange	0.985	Yes
sport car	sport car	0.966	Yes

My images

Table 2: My seven images, centre crop. I wrote the expected class for the four photos before my first run on 12 November, and for the three synthetic images after seeing results from an earlier trial on 4 November.

Image	Group	I expected	Predicted	Score	Next two
Cat sitting, seen from behind	outside	red panda	espresso	0.877	koala 0.051, ladybug 0.031
Cat lying on a blanket	outside	red panda	espresso	0.916	red panda 0.035, pizza 0.029
Cat curled in a plant pot	outside	koala	espresso	0.795	ladybug 0.094, red panda 0.063
New Year dinner table	outside	school bus	koala	0.466	espresso 0.250, lifeboat 0.114
Synthetic sports car	trained	sport car	ladybug	0.876	bell pepper 0.057, orange 0.051
Synthetic teacup	related	espresso	espresso	0.974	ladybug 0.016, orange 0.010
Synthetic wrench	outside	ladybug	espresso	0.983	red panda 0.008, ladybug 0.005

Preparation

Table 3: Top class and score under each preparation. The last column is the mean absolute difference per pixel value (0 to 255) between the centre-crop and browser-like inputs.

Image	Centre crop	Padded	Browser-like	Difference
Cat sitting	espresso 0.877	espresso 0.872	koala 0.745	9.8
Cat on blanket	espresso 0.916	espresso 0.796	pizza 0.886	22.4
Cat in plant pot	espresso 0.795	espresso 0.621	koala 0.851	22.2
Dinner table	koala 0.466	sport car 0.337	sport car 0.536	15.7
Sports car	ladybug 0.876	ladybug 0.898	ladybug 0.861	14.0
Teacup	espresso 0.974	espresso 0.983	espresso 0.987	5.3
Wrench	espresso 0.983	espresso 0.978	espresso 0.714	8.7

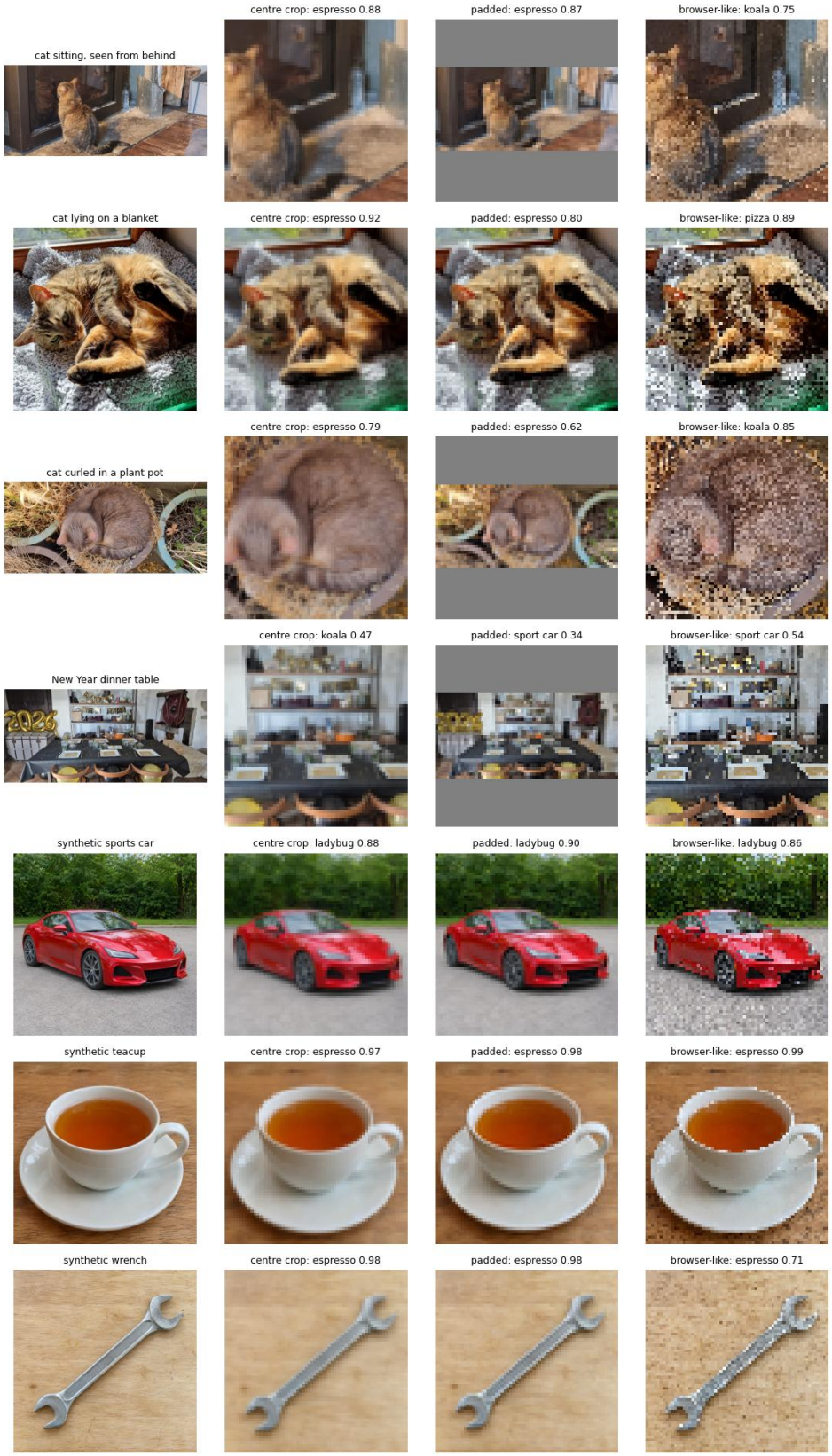


Figure 1: Each image and its three 64 × 64 inputs, with the top class and score. From my notebook run.

Threshold

Table 4: Images a minimum score would accept, compared at full precision. Built-in examples kept, of 10; images outside the label set accepted, of 5, under each preparation.

Threshold	Built-in kept	Outside, centre crop	Outside, padded	Outside, browser-like
0.50	10	4	4	5
0.60	10	4	4	4
0.70	10	4	3	4
0.80	9	3	2	2
0.88	7	2	1	1
0.90	7	2	1	0
0.95	7	1	1	0

Layers

Table 5: Share of positive values after each ReLU layer, averaged by group (centre crop for my images). The measure ignores how large the values are and where they sit.

Group	relu_1_1	relu_1_2	relu_2_1	relu_2_2
Built-in examples (10)	0.833	0.598	0.790	0.618
Outside the label set (5)	0.879	0.593	0.833	0.597
Trained class: sports car (1)	0.861	0.654	0.784	0.617
Related class: teacup (1)	0.865	0.559	0.787	0.576

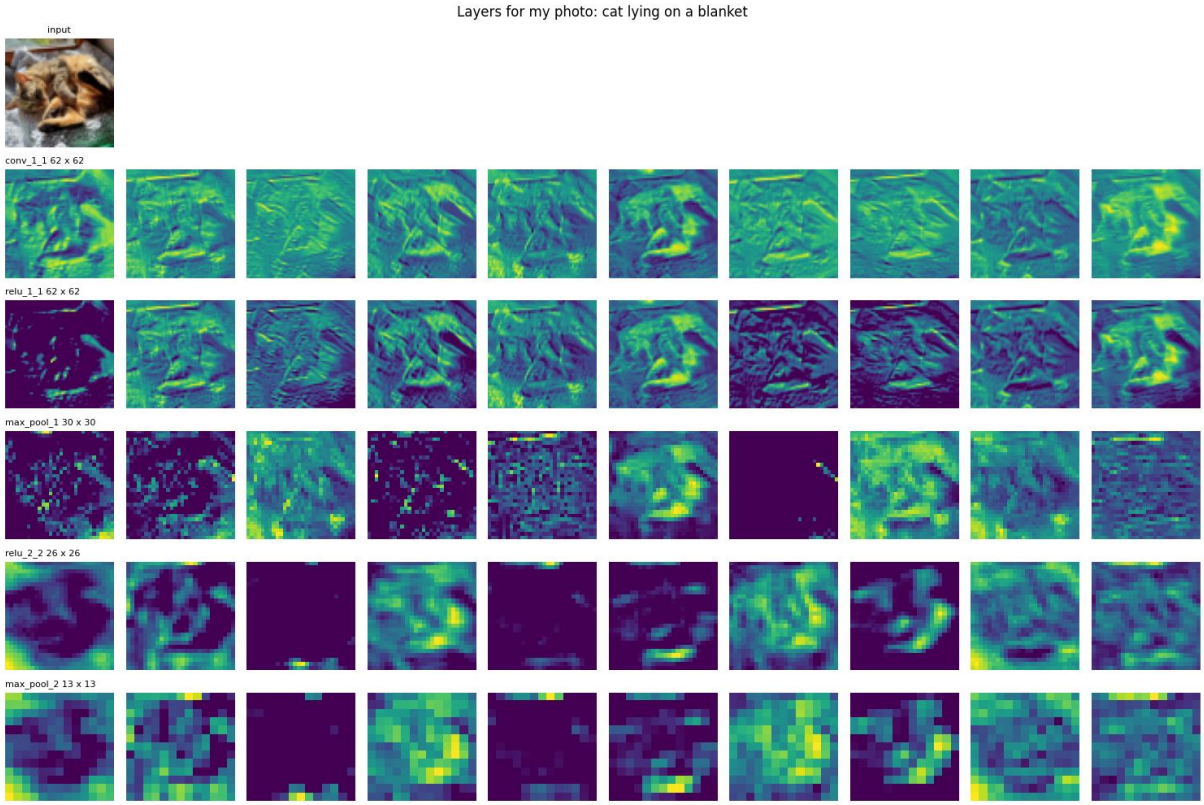


Figure 2: The ten maps after five layers for the cat lying on a blanket, from the first convolution to the second pooling layer. From my notebook run.

Browser and Colab

Table 6: Top class in CNN Explainer, from my screenshots of the original files, against my Colab run. The browser-like resize copies only the tool's resize step.

Image	**Shape**	**Browser**	**Colab, centre crop**	**Colab, browser-like**
Cat sitting	wide	espresso	espresso	koala
Cat on blanket	square	pizza	espresso	pizza
Cat in plant pot	wide	espresso	espresso	koala
Dinner table	wide	koala	koala	sport car
Sports car	square	ladybug	ladybug	ladybug
Teacup	square	espresso	espresso	espresso
Wrench	square	espresso	espresso	espresso

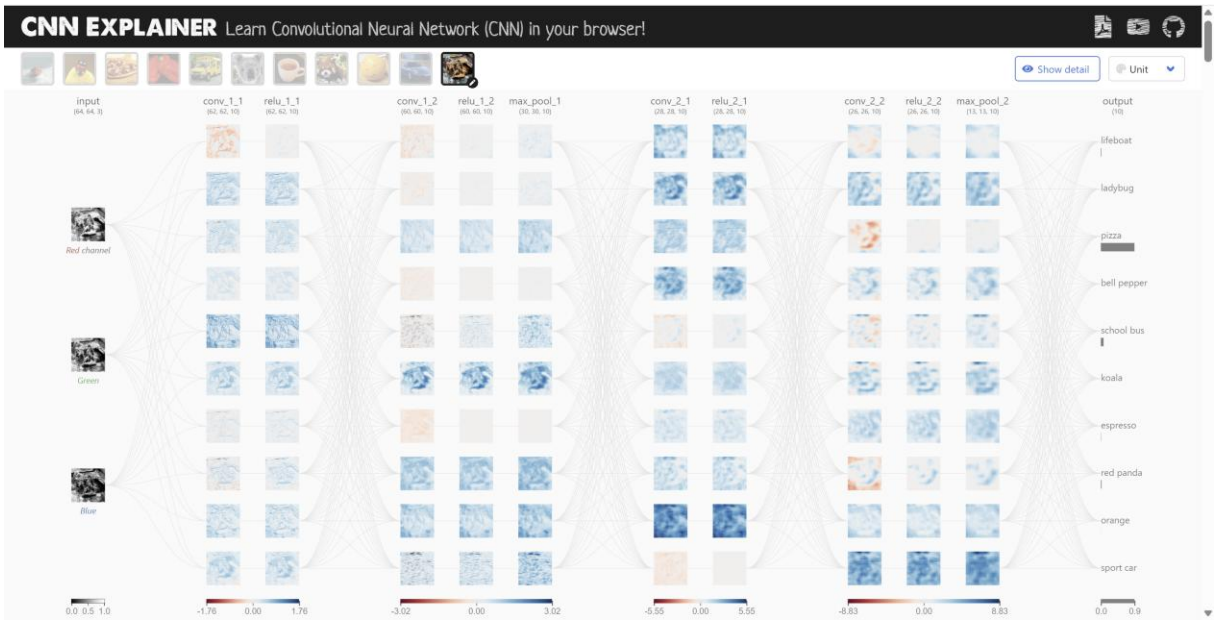


Figure 3: CNN Explainer overview for the cat lying on a blanket, from the input channels to the ten output classes, with pizza highest. My screenshot.

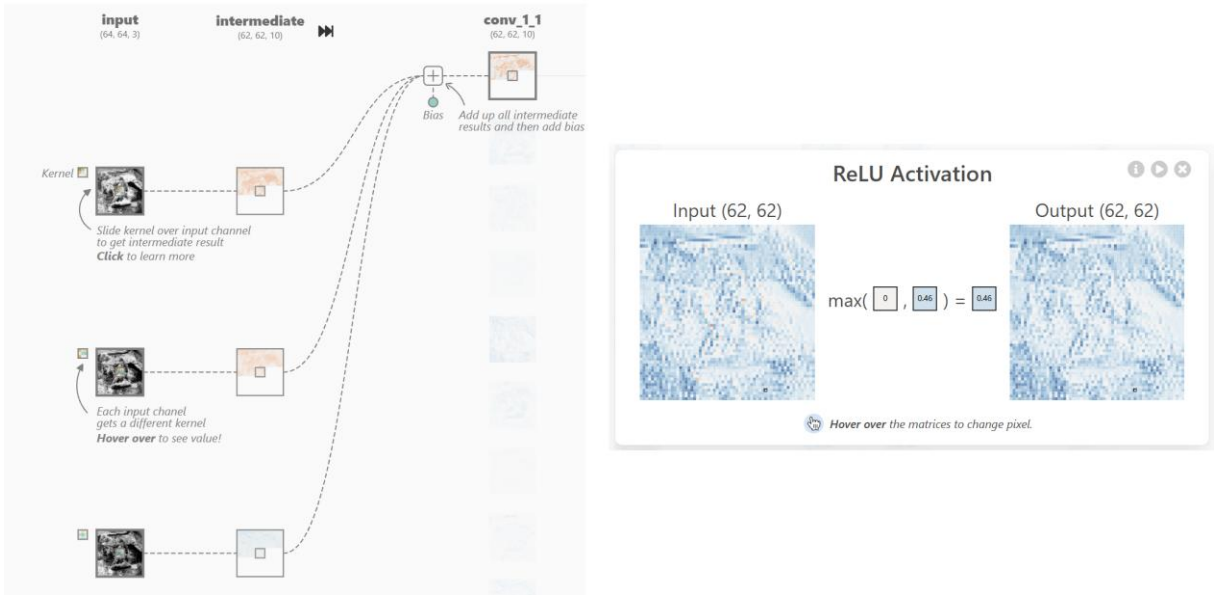


Figure 4: CNN Explainer detail views for the same photo: one map of the first convolution layer built from its three input channels (left), and one ReLU map, where max(0, x) keeps positive values (right). My screenshots.

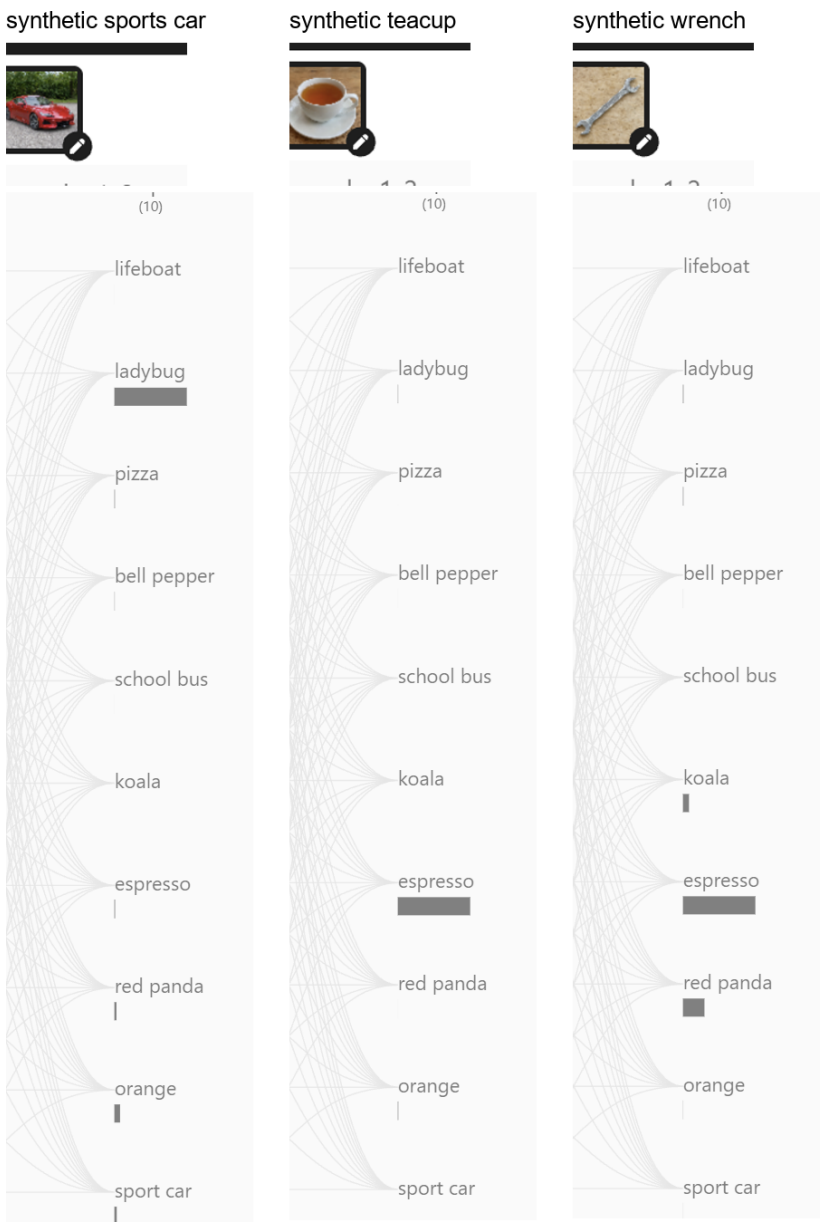


Figure 5: CNN Explainer output layer for the three synthetic images. My screenshots.

Unit 9 and Unit 10

Table 7: The two activities answer different questions. Unit 9 figures come from my Unit 9 run of 11 November 2026.

Property	**Unit 9**	**Unit 10**
Task	Train and test a CNN	Inspect a trained CNN
Input	32 × 32 RGB	64 × 64 RGB
Classes	CIFAR-10, including cat	Ten Tiny ImageNet classes, no cat
Parameters	225,610	19,920
Evidence	10,000 test images, accuracy 0.639	10 built-in examples and 7 chosen images

References

Géron, A. (2025) *Hands-On Machine Learning with Scikit-Learn and PyTorch*. Santa Rosa: O'Reilly.

Krizhevsky, A. (2009) *Learning multiple layers of features from tiny images*. Technical report. Toronto: University of Toronto. Available at: https://cave.cs.toronto.edu/kriz/cifar.html (Accessed: 11 November 2026).

Polo Club of Data Science (2023) *cnn-explainer* (commit d0971f9) [Source code]. Available at: https://github.com/poloclub/cnn-explainer/tree/d0971f9447ed9806022a3d47587b62394682bc51 (Accessed: 12 November 2026).

Wang, Z.J., Turko, R., Shaikh, O., Park, H., Das, N., Hohman, F., Kahng, M. and Chau, D.H. (2021) 'CNN Explainer: learning convolutional neural networks with interactive visualization', *IEEE Transactions on Visualization and Computer Graphics*, 27(2), pp. 1396–1406. Available at: https://doi.org/10.1109/TVCG.2020.3030418.

Yang, J., Zhou, K., Li, Y. and Liu, Z. (2024) 'Generalized out-of-distribution detection: a survey', *International Journal of Computer Vision*, 132(12), pp. 5635–5662. Available at: https://doi.org/10.1007/s11263-024-02117-4.

Supplied materials: University of Essex Online Unit 10 seminar brief, CNN Tutorial.

AI Acknowledgement

Codex was used to assist in identifying appropriate code solutions and to proofread and refine the code I produced. Claude and Grammarly supported proofreading, language refinement, and formatting of my written text. All final decisions, analyses, interpretations, conclusions, and recommendations are my own.